

Enterprise Bulk Import System Design

Research reference · 2026-08-24

Enterprise Bulk Import System Design — Research Notes

Goal: understand how enterprises handle large-scale imports (multiple entity types, nested structures) so we can leverage the patterns in our own import work.

Compiled Aug 2026. Sources listed at bottom.

TL;DR

Every mature import system converges on the same skeleton:

Upload → Stage (raw landing) → Parse/Stream → Validate → Resolve keys/order → Load in batches

The three hard problems are always the same:

1. **Ordering** — child rows arrive before parents exist (FK violations).
2. **Failure semantics** — what happens when row 40,000 of 200,000 fails?
3. **Scale** — never hold the whole file in memory; batch writes; manage lock contention.

Vendors (Salesforce, Shopify) solve these with **async job models + flat line-oriented payloads + per-line error files**. Internal systems solve them with **staging layers + dependency-ordered loads + idempotent upserts + workflow orchestration**.

Part A — How platform vendors expose bulk imports

1. Salesforce Bulk API 2.0 (the canonical async job model)

Model: Create job → upload one CSV → server auto-splits into batches → poll status → download success/error result files.

Key design decisions:

Decision	Detail
Async job state machine	Open → UploadComplete → InProgress → Completed/Failed/Aborted . Jobs expire after 24h open / 7d retention of results
Auto-batching	Server splits your CSV into ~10,000-record batches automatically (Bulk API 1.0 made the client do it)
Throughput ceiling	15,000 batches/day × 10k records = 150M records/day ; <2,000 records → use synchronous REST instead
PK chunking	For huge queries: split by record-ID boundaries, default 100k, max 250k rows/chunk; supports restart via <code>startRow</code> after mid-job failure
Parallel vs serial mode	Parallel = fast but risks lock contention (two batches touching the same parent account). Serial = slow but safe. Salesforce explicitly says: reorganize batches by parent ID before falling back to serial mode
Error semantics	Batch completes ≠ all rows succeeded. Per-record results in separate success/failure/unprocessed files . Failed rows must be resubmitted manually
Retry	Server retries transient failures (up to 10–20×); client resubmits genuinely failed records

Pros: battle-tested at extreme scale; clean separation of submission and processing; error files make partial failure manageable; automatic batching removes client complexity.

Cons: eventual consistency only (polling); no cross-record transactions (partial success is the norm); lock contention forces manual batch organization or slow serial mode; CSV-only in 2.0 (no nested objects).

2. Shopify GraphQL Bulk Operations (the canonical nested-data model)

Model: Upload a JSONL variables file → run any mutation once per line asynchronously → download JSONL results.

Key design decisions for **nested structures**:

Decision	Detail
Flat-line encoding	One line = one input unit, <i>no matter how complex</i> (product + variants encoded inline as nested JSON on a single line)
Flattened output with <code>__parentId</code>	Query results flatten trees into lines: each product followed by its variants, each variant followed by its images. Children reference parents via an injected <code>__parentId</code> field. Client reconstructs the tree while streaming
Depth limits	Max 2 levels of nested connections, max 5 connections per query — they cap tree complexity explicitly

Decision	Detail
No ordering guarantees	"Avoid relying on a particular sequence of lines" — each line executes independently
Per-line isolation	Each line validated/executed independently; errors reported inline next to successes in the result file; <code>partialDataUrl</code> gives you whatever completed before a failure
Concurrency caps	Up to 5 concurrent bulk ops per app/shop (2026-01+); mutation ops must finish within 24h; input file $\leq 100\text{MB}$

Pros: elegant solution to nested hierarchies over a line-oriented transport; per-line error isolation; zero rate-limit cost for the bulk execution itself; streaming-friendly (parse one line at a time).

Cons: depth caps mean very deep trees need multiple passes; no ordering control (client must handle parent-before-child itself); 24h hard timeout pushes complexity back to the caller.

3. The shared vendor playbook

Regardless of vendor, the pattern is identical:

1. **Never process synchronously above a small threshold** (Salesforce: 2,000 records).
2. **Client uploads one flat file; server owns partitioning.**
3. **Results come back as downloadable files**, not API responses (error reports can be huge — presigned URLs, not JSON arrays).
4. **Per-row/per-line error isolation** — one bad row never kills the job.
5. **Explicit resource ceilings** — daily quotas, concurrent-job caps, timeouts — published upfront.

Part B — Internal architecture patterns

4. The staging layer (why almost everyone has one)

Four-stage staging pattern (from production write-ups):

```

Stage 1: raw landing      - byte-exact copy of uploaded file (audit/replay)
Stage 2: normalized stage - typed columns, standardized source keys, validation flags,
Stage 3: key resolution   - map source business keys → target IDs; find unresolved FK r
Stage 4: final load       - insert parents → children → junction tables, then reconcile

```

Pros: separates "can we read the file?" from "is the data relationally valid?"; enables dry-runs and reconciliation before anything touches prod; full auditability and replayability; reject quarantine.

Cons: extra storage and pipeline stages; more moving parts; latency between upload and availability; teams sometimes abuse deferred constraints instead of real ordering design.

When you can skip it: single entity type, clean source keys, controlled upstream. **When you must have it:** messy customer files, out-of-order arrival, key remapping, audit requirements.

5. Multiple entity types in one import — ordering by dependency depth

The standard mental model is **load by dependency depth** (a topological sort of entity types):

```
Depth 0: independent reference tables
Depth 1: parent entities
Depth 2: child entities
Depth 3: grandchild/detail rows
Last:    junction/bridge tables (need BOTH sides resolved first)
```

Template: references → parents → children → details → junctions → FK reconciliation checks → promote batch only if validation passes.

Two implementation styles:

Style	Mechanism	Pros	Cons
Static layering	Hardcode type order (types → subtypes → instances)	Simple, predictable, easy to reason about	Brittle when graph changes; underutilizes parallelism within layers
DAG/topological sort	Build dependency graph from schema, sort at runtime	Adapts to new types automatically; maximizes intra-layer parallelism	More infrastructure; cycles must be detected and rejected early

Cross-file/cross-type referential integrity must be checked *before* persistence: orphaned children get actionable errors ("line item references unknown Order X"), not FK exceptions at insert time.

6. Nested/hierarchical structures (BOMs, org charts, category trees)

This is the hardest part. Four approaches seen in practice:

Approach	How it works	Pros	Cons
Flatten + parent refs (Shopify style)	Encode tree as flat lines with <code>__parentId</code> ; reconstruct while streaming	Streaming-friendly; bounded memory; works over any queue/file transport	Client must rebuild tree; depth caps often imposed; two representations of truth
Two-pass import	Pass 1: validate everything, build ID map, detect orphans/cycles. Pass 2: write in dependency order	Catches structural errors before ANY write; correct FK resolution	Double processing cost; needs staging area between passes
Recursive/level-by-level resolution	Process level 0, resolve IDs, process level 1 referencing resolved IDs, repeat	Natural fit for self-referencing hierarchies (BOM assemblies); incremental progress visible	Deep trees = many sequential rounds; tail latency dominated by max depth
Deferred constraints / disable FK during load	Insert everything unordered, fix references after, re-enable constraints	Fast; no ordering logic needed	Dangerous; one bad row poisons silently; requires strong post-load reconciliation; widely considered a last resort

Cycle detection is mandatory for self-referencing structures (a BOM part containing itself): detect during validation pass via visited-set traversal, reject with a precise path (`A → B → C → A`), don't let it infinite-loop a worker.

Practical bound: production systems impose explicit depth limits (Shopify: 2 connection levels) not because trees can't be deeper but because unbounded recursion breaks memory, timeouts, and error reporting.

7. Failure semantics — the decision matrix

Strategy	Behavior	Use when
Fail-fast	Stop at first error	Strict financial/legal batches where partial data is worse than none
Skip-and-continue (most common default)	Record error, keep going; report at end	Best-effort customer imports; most rows are good
Collect-all	Process every row, return every error	Diagnostic/dry-run/audit modes
Partial commit + correction file	Commit valid rows; emit failed rows + <code>_error</code> column as CSV for user to fix and re-upload	Consumer-facing imports; closes the loop without support tickets

Supporting machinery:

- **Idempotent upserts:** derive deterministic row IDs (`import_id + row_index`), unique-index them, `ON CONFLICT DO NOTHING / upsert` — makes retries safe after crashes.
- **Checkpoint/resume:** persist `last_committed_row_index` after each flush; on retry skip ahead. (Temporal formalizes this with activity heartbeats + event-history replay.)
- **Dead-letter queue:** permanently failed batches land somewhere inspectable rather than vanishing.
- **Saga/compensation:** for multi-step flows where rollback matters (create entities → link relations → notify); compensating actions undo step N on failure at N+1. Temporal's sweet spot.
- **Transactionality reality check:** true all-or-nothing across 200k rows is usually *not* attempted — the industry norm is per-batch atomicity + job-level partial-success reporting.

8. Validation strategy

Three distinct error classes, handled differently:

1. **Structural** — missing column, wrong delimiter, bad encoding → reject whole file immediately, before any parsing work.
2. **Row-level** — bad type, missing required field → collect per-row, feed the correction-file loop.
3. **Relational/business** — FK target doesn't exist, duplicate business key, cycle in hierarchy → needs cross-row/cross-file context; do it in the validation pass with preloaded ID sets (set-membership, NOT per-row queries — per-row FK lookups serialize your import to single-digit rows/sec).

Dry-run mode = same code path as execute, transaction rolled back at the end. Cheap to build, huge trust win.

Scale note for FK checks: preload valid IDs into memory up to ~100k entries (10–20MB); beyond ~10M, spill to a temp table and check per batch with `NOT IN (SELECT ...)`.

9. Scale techniques checklist

- **Stream everything** — streaming parser emits rows; only current batch (~500 rows) + accumulated errors in memory. 1M-row file ≈ same memory as 1k-row file.
- **Batch DB writes** — ~500 rows/batch is the practical sweet spot for Postgres-style `INSERT ... VALUES`; >1000 shows diminishing returns and lock contention.
- **Use COPY/bulk loaders** for initial landing into staging (orders of magnitude faster than INSERT).
- **Parallelize within dependency layers**, never across them.
- **Organize batches by parent key** to avoid lock contention (Salesforce's AccountTeamMember lesson: group rows touching the same parent into one batch).

- **Backpressure:** workers publish batches to a stream; a dedicated writer consumes at DB-safe rate — decouples worker scaling from write throughput.
- **Progress tracking:** counter in Redis/job row updated every N batches; UI polls or subscribes.
- **Error reports as files:** write failure CSV to object storage, return presigned URL — keeps API responses bounded regardless of failure count.
- **Per-tenant concurrency limits** in the queue so one big importer can't starve everyone.

Part C — Orchestration engines for import jobs

	Kestra	Temporal	Airflow	Camunda
Model	Declarative YAML workflows	Durable code (Java/Go/Python/TS)	Python DAG scheduler	BPMN process diagrams
Durability	State tracked across restarts; restart-from-failed-task	Full event history + deterministic replay; survives anything	Task retries from scratch; external state (XCom/DB)	Persistent process instances
File/data handling	First-class (auto S3/GCS upload/download between tasks, 1400+ plugins)	Manual — you manage I/O, pass pointers not data	Via providers/operators	Via service tasks/integrations
Sweet spot	Data pipelines, ETL, file-driven imports, mixed eng/analyst teams	Mission-critical multi-step business flows, sagas, long-running (days–weeks)	Scheduled batch ETL on cron	Human-in-the-loop approval flows
Weakness for imports	Complex dynamic branching logic harder than code	Learning curve (~1 month), throughput overhead, manual file plumbing	No durable mid-task state; sensors occupy slots	Heavyweight for pure data movement
Cost signal	Open-source friendly	Cloud premium meaningful (~\$28k/yr vs \$8k MWAA in one fintech case)	Cheapest managed option	Mid

Real-world anchors:

- Fintech chose **Temporal** for a 2.4B-row, 18-day replayable ledger migration precisely because deterministic replay + audit trail justified cost; kept Airflow alongside for regular ETL.

- Enterprise PII ingestion pipeline chose **Temporal** because "no complaint may ever be silently lost" + resume-from-last-row heartbeats were requirements, not nice-to-haves.
- Teams routinely run **both**: declarative orchestrator for data movement, durable-execution engine for transactional steps.

Rule of thumb: **if the import is mostly file→transform→load, a declarative orchestrator wins. If it's multi-service transactions with compensation and human gates, durable execution wins.**

Part D — Leverage for our project (multi entity types + nested structures)

Mapping the research onto our shape (import of multiple entity types with nested parent-child structures, e.g., BOM-like trees):

Recommended architecture



- ⑤ Reconcile: counts vs manifest, orphan scan, totals
Error report → object storage CSV (failed rows + `_error` column)
Status: COMPLETED or PARTIAL with presigned download

Decisions cribbed directly from the research

1. **Async job + status polling from day one** (Salesforce threshold: sync only under ~2k records). Never let an HTTP request own a large import.
2. **Flat internal representation with parent refs** (Shopify `__parentId`) as the canonical interchange format between pipeline stages — streams through queues/files with bounded memory; reconstruct hierarchy only where needed.
3. **Two-pass for nested types**: validate + resolve ALL keys before writing anything. Single-pass fails FK checks for every child whose parent appears later — the classic silent mass-failure.
4. **Level-by-level loading for deep trees** with explicit max-depth rejection + cycle path reporting.
5. **Skip-and-continue default, fail-fast opt-in, correction-file round-trip** for end users; dry-run endpoint sharing the exact execute code path.
6. **Idempotency everywhere**: `(import_id, row_index)` unique keys + upsert semantics means crash-retry is free.
7. **Orchestration choice**: we already run Kestra — it fits the file-driven, multi-stage shape natively (per-task retries, restart-from-failure, file passing between tasks). Reserve durable-execution (Temporal-style) thinking only for steps that span multiple services with compensation needs.
8. **Publish explicit ceilings** like the vendors do: max file size, max rows, max depth, concurrent-jobs-per-tenant. Undocumented limits become 2am incidents.

Anti-patterns to avoid (documented failure modes)

- Per-row FK existence queries during load (serializes throughput).
- Disabling constraints as the ordering strategy without reconciliation plan.
- Loading junction/mapping tables before both sides exist.
- Relying on file row order for correctness (Shopify explicitly forbids it; so should we).
- Returning error arrays in API responses instead of files.
- Unbounded recursion on hierarchical data (stack overflow / runaway workers).

Sources

- Salesforce: [PK Chunking](#) · [Bulk API Limits](#) · [Data Load Guidelines](#) · [Bulk API Guide PDF](#) · [2.0 vs 1.0](#)

- Shopify: [Bulk Imports](#) · [Bulk Queries](#) · [bulkOperationRunMutation](#)
- Pipelines: [Production-safe spreadsheet import pipeline](#) · [FK & CSV load-order templates](#) · [Tenant data import system at scale](#) · [CSVBox relational imports](#) · [Import pipeline components](#) · [Granit import pipeline](#)
- Orchestration: [Why Temporal for a data pipeline](#) · [Kestra vs Temporal](#) · [Kestra/Temporal/Prefect in production](#) · [Temporal vs Airflow](#) · [Fintech ETL replay case study](#)